Evidence-backed Authorization Decision Analysis for SAGA

Abstract

Policy-governed agent communication requires authorization decisions to preserve intended policy semantics across evaluation and enforcement. Authentication and confidentiality alone do not establish this consistency. We investigate the requirement through a SAGA case study. Archived direct calls to the unmodified contact-policy matcher demonstrate an order-dependent authorization decision inconsistency: a specific-deny/general-allow rulebook yields an allow-side budget although documented most-specific semantics select denial. Separately, a diagnostic extension retains the queried authentication correspondences and token secrecy while an added authorization correspondence is false. This identifies a verification-coverage gap, not a formal proof of the matcher inconsistency. For the recorded policy input, an independent finite specification evaluator derives Deny, while the archived matcher observation supplies Allow. A decision bridge checks the corresponding recorded policy input and observation provenance, constructs the decision pair, and reconstructs the fixed authorization-divergence scenario analyzed in the archived ProVerif 2.05 run. Under its explicit symbolic assumptions, the derived model admits application-message acceptance (`ChatAccept`) and establishes that acceptance implies prior token issuance (`TokenIssue`). Independent gate controls distinguish Provider and receiver blocking stages. These results connect implementation observations to symbolic consequences through checked evidence transfer for the recorded case. They establish neither cryptographic compromise nor deployed-service reproduction. Mapping the concrete evaluation to symbolic participants remains an explicit abstraction; general implementation-to-model refinement and correctness for arbitrary policies remain unproved.

1 Introduction

As agentic systems increasingly rely on autonomous agent-to-agent communication, they require identity verification, secure communication, and

contact policies. Authentication answers "who is communicating," whereas authorization asks whether that identity is permitted to communicate. Authenticating a participant and protecting its messages do not establish that the receiving user's policy permits the interaction.

We study *authorization decision consistency* through evidence-backed analysis of a recorded SAGA [12] case. Our research question is how implementation observations and specification-side evaluation can identify authorization decision inconsistencies, and how their protocol consequences can be analyzed under explicit symbolic model assumptions. This connects policy meaning, observed implementation decisions, and modeled actions that consume those decisions. Chapter 3 presents the broader requirement that accepted contact be justified by an applicable decision as an ideal objective. We analyze a specific class of decision inconsistencies and their conditional consequences, without proving the complete decision-to-access justification relation or authorization correctness in deployed executions.

The verification-coverage gap motivates this analysis. In the Stage 1 diagnostic extension, the original communication model's three authentication correspondences and token secrecy remain true, while the added correspondence from token receipt, `Accept`, to `PolicyAllow` is false. The process never emits `PolicyAllow`; its `PolicyDeny` event records denial without guarding execution. This diagnostic exposes an authorization obligation absent from the original property set. It does not formally detect the implementation matcher inconsistency, and its authentication and secrecy results are not automatically inherited by later consequence models.

We compare implementation observations with specification-side evaluation. Archived direct function-level calls to the unmodified matcher show that a specific denial followed by a general permission returns budget 10, whereas reversing the same rules returns -1. Documented most-specific semantics require denial in both orders. An independent finite specification evaluator selects the specific denial for the recorded ordered policy input; the archived implementation observation supplies permission. This is an order-dependent authorization decision inconsistency at the matcher boundary, not an observation of a deployed service accepting communication.

An evidence-backed analysis procedure connects that comparison to symbolic consequence analysis. Its decision-bridge component combines independent specification evaluation, observation validation, provenance checking, and scenario reconstruction. It checks the independently derived Deny decision and observed Allow decision against the recorded ordered rules, matched identity, and historical source provenance, then reconstructs the scenario region of a derived ProVerif model. Within the fixed Deny/Allow

adapter, this avoids manual transcription of the decision values for each run; the protocol template and symbolic role mapping remain analysis choices. The archived ProVerif 2.05 execution reports `ChatAccept` reachable and establishes that acceptance implies prior `TokenIssue` under the model assumptions. ProVerif consumes fixed symbolic facts and does not execute Python. This is checked evidence transfer, not a general refinement proof. The earlier preset-divergence model retains its separate historical provenance.

The procedure also includes consumer-side propagation validation in a controlled environment. Fresh observations of the production matcher inside the Provider and receiver consumers are linked through the real Provider `/access` route and receiver handshake to token issuance and storage. Allow/allow, deny/deny, and deny/allow specification/implementation controls exercise this bounded workflow with transport and registration fixtures. Execution stops at the conversation boundary; this experiment does not validate message acceptance or identify the historical symbolic witness with a concrete service session.

As complementary experiments, we compare where independent gate controls block execution. Provider denial blocks material release and downstream events. Receiver denial blocks issuance and acceptance while release remains reachable. These results locate blocking stages under the modeled gate and handoff assumptions; they do not validate a deployed repair. Together, the analyses distinguish decision correctness, evidence transfer, and the consequences of consuming a decision.

We make the following case-study contributions:

- **Authorization decision inconsistency analysis.** For the recorded SAGA case, we identify and analyze an order-dependent disagreement between the specification-side decision and the implementation-observed decision, using independent finite evaluation and archived direct calls to the studied matcher.

- **Evidence-backed analysis procedure.** We connect specification evaluation, observation validation, provenance checking, and scenario reconstruction in a reproducible procedure for the recorded case. The decision bridge implements these analysis steps for a fixed consequence template with a Deny/Allow input. Complementary consumer-side propagation validation links fresh matcher observations to production consumer execution through token issuance and storage in a controlled environment, stopping before message acceptance. The procedure is not a general framework or a verified compiler.

- **Formal consequence analysis.** We use ProVerif to analyze conditional consequences of the recorded decision divergence in the derived symbolic model: message acceptance is reachable and implies prior token issuance. Complementary independent gate controls identify the modeled release and issuance boundaries; they do not validate a deployed repair.

Mapping the concrete evaluation to symbolic protocol roles remains an explicit modeling abstraction. The evidence does not establish general Python-to-ProVerif refinement, arbitrary-policy correctness, or the same concrete service session across implementation and model. It demonstrates neither cryptographic compromise nor a deployed-service exploit.

2 Background

2.1 Agentic Systems and Inter-Agent Communication

Large language model (LLM)-based agents can interact with other agents to coordinate work and delegate tasks on behalf of users. Such interaction allows a task to involve multiple participants, each contributing information or performing part of the work. Communication may proceed autonomously, with limited human involvement in individual exchanges. In this setting, delegation describes assigning work to another agent; the permission to contact that agent is governed by the communication system's authorization policy.

Inter-agent communication therefore involves both task messages and mechanisms for governing their exchange. Agent identities identify the participants, secure transport protects communication, and authorization policies specify which contacts are permitted. We call the participant requesting contact the *initiating agent* and the participant receiving that request the *receiving agent*.

SAGA provides a concrete setting for these roles. Users register their agents with a Provider that maintains agent contact information and user-defined access-control policies. The Provider assists agents in enforcing these policies, while access-control tokens mediate subsequent communication. This separates the user's configuration of contact permissions from the agents' autonomous exchange of task messages.

2.2 Authentication, Authorization, and Confidentiality

Authentication answers "Who is communicating?" It establishes the identity of a participant in a protocol exchange. Identity credentials and checks on protocol messages provide the basis for associating an exchange with that participant.

Authorization answers "Who is allowed to communicate?" It determines whether a participant has permission to perform the requested interaction under an applicable policy. For contact authorization, the relevant decision concerns an initiating agent's permission to communicate with a receiving agent according to the receiving user's rules.

Confidentiality answers "Who can learn the exchanged information?" It concerns restricting disclosure of protected information to the parties permitted to learn it within the specified security model. Protected information may include communication content and credentials exchanged by the protocol.

These properties are complementary. Authentication supplies the participant identity to which a policy can apply; authorization determines the permitted interaction; confidentiality protects information exchanged during the protocol. Each describes a distinct aspect of communication security.

2.3 Policy-Based Authorization

Policy-based access control expresses permissions through rules and applies those rules when handling requests. Three concepts organize this process: policy specification, policy evaluation, and enforcement.

Policy specification defines the meaning of the rules, including which subjects and interactions they cover and how applicable rules determine a decision. In SAGA, a receiving user's contact rulebook associates agent-identity patterns with budget values. Pattern matching identifies applicable rules; the policy's rule-selection convention and interpretation of the selected budget determine contact permission.

Policy evaluation applies those semantics to a particular request. Its inputs include the requesting identity, the target, and the applicable policy. The result expresses whether the request is permitted under those inputs. Policy meaning defines the decision to be computed, while the evaluation procedure implements that computation.

Enforcement applies the decision at the relevant protocol boundary. In SAGA's communication flow, the Provider evaluates the receiving user's stored rulebook before releasing the material needed to continue contact. The receiving agent performs policy and credential checks, issues an access-control token, and subsequently checks a presented token when handling a message. These stages distinguish permission to proceed, credential issuance, and message acceptance.

The authorization outcome thus depends on the specified policy semantics, their evaluation for the request, and the application of the resulting decision at the appropriate stages. A policy decision concerns a particular initiating agent and receiving agent; where a credential mediates contact, its use is part of this authorization context. This terminology provides the basis for the policy–authorization consistency requirement developed next.

3 Security Gap and Problem Formulation

3.1 Authentication Is Not Authorization

SAGA uses identity checks and access-control tokens to govern contact between agents. These mechanisms answer different questions. Authentication establishes which participant completed a protocol step; token validation establishes that a presented token satisfies the checks performed by the receiving agent. Authorization additionally asks whether the receiving agent's user-defined contact policy permits the initiating agent to contact it. A credential can be authentic and valid under the token checks while the decision to make that credential available disagrees with the policy's intended meaning.

The reference communication model focuses on participant authentication and token secrecy. These guarantees concern the origin of protocol messages and the confidentiality of credentials; they do not, by themselves, establish that a contact complies with the receiving user's policy. In particular, confidential delivery to an authenticated participant does not establish that the participant is entitled to receive a credential under that policy.

The missing requirement is an authorization basis for accepted contact, preserved through credential issuance and message acceptance. Chapter 4 examines implementation disagreements with policy meaning. This chapter defines the requirement without asserting an end-to-end access violation.

3.2 Policy–Authorization Consistency

As an ideal authorization objective motivating this study, we call the end-to-end requirement *policy–authorization consistency*: every accepted access must be traceable to an applicable authorization decision. The ideal requirement ranges over every execution and accepted-access event e:

$$\mathsf{AccessAccepted}(e) \implies \exists d.\ \mathsf{AllowDecision}(d) \wedge \mathsf{Justifies}(d, e).$$

In addition to this end-to-end property, we consider authorization-soundness conditions at intermediate protocol boundaries, including Provider grants and credential issuance. These intermediate events are not themselves instances of AccessAccepted.

Here, d is a decision occurring in that execution, and $\mathsf{AllowDecision}(d)$ means that it grants permission under the intended policy semantics. An implementation's report of "allow" alone does not establish this predicate. The relation $\mathsf{Justifies}(d, e)$ requires that the decision precede the accepted

access, cover the same subject, target, and, where applicable, operation, and remain applicable under the relevant policy state. Where a credential mediates access, the decision must also be linked to the grant and credential used for that access. An unrelated earlier allow decision is not a sufficient justification. This is a general requirement whose concrete events and policy-state conditions must be fixed for each formal model.

This requirement motivates the analysis; it is not a theorem established by this paper. We do not attempt to prove the complete Justifies relation or the authorization correctness of SAGA as a whole. We study a specific class of authorization decision inconsistencies: disagreement between a specification-side decision and an implementation-observed decision for the recorded policy input and matched identity. The subsequent protocol analysis concerns conditional consequences of that divergence under explicit symbolic model assumptions, not authorization correctness in deployed executions.

Let I be an initiating agent, R a receiving agent, and P_R the contact rulebook selected by R's user. Each rule associates an agent-identity pattern with a budget. We write $\mathsf{Allow}_{\mathrm{spec}}(P_R, I)$ for the decision prescribed by the intended policy semantics, rather than the value returned by any particular implementation of the matcher. In the cases considered here, the unique highest-scoring matching rule determines the budget; a positive budget permits contact, whereas a nonpositive budget or no matching rule does not. Rule selection follows the matcher's documented most-specific-rule contract and its AID specificity score; the positive-budget convention follows the Provider's access check.[1] For the symbolic analysis, we fix the rulebook throughout an execution and consider either no matching rule or a unique highest-scoring match. Equal-score conflicts require an explicit convention and are outside this initial scope.

We distinguish three analysis-level events. $\mathsf{ProviderGrant}(I, R)$ denotes the Provider's release, through its access interface, of the material needed for I to continue the contact protocol with R. The event $\mathsf{TokenIssued}(R, I, T)$ denotes R's creation of token T for I and its release for delivery through the protocol; it does not assert that I has received or used T. Finally, $\mathsf{MessageAccepted}(R, I, T, m)$ denotes R's acceptance of an application message m from I with token T, after the receiving agent's access checks, for application processing. It does not assert the completion of a downstream tool action.

[1] See `match()` and `aid_specificity()` in `saga/common/contact_policy.py`, and `access()` in `saga/provider/provider.py`.

Under this static-policy scope, the intermediate condition at the Provider boundary is

$$\mathsf{ProviderGrant}(I, R) \implies \mathsf{Allow}_{\mathrm{spec}}(P_R, I),$$

whereas a necessary permission condition for end-to-end consistency at the final message-acceptance boundary is

$$\mathsf{MessageAccepted}(R, I, T, m) \implies \mathsf{Allow}_{\mathrm{spec}}(P_R, I).$$

The intermediate condition at token issuance is

$$\mathsf{TokenIssued}(R, I, T) \implies \mathsf{Allow}_{\mathrm{spec}}(P_R, I).$$

All three implications range over every occurrence of the respective events and all their arguments. The intermediate conditions are separate obligations, not logical consequences of end-to-end consistency alone. These implications express permission at their respective boundaries; they do not replace the decision-to-access linkage required by Justifies. Nor does policy permission alone suffice for access: certificate checks, key availability, token validity, expiry, and quota may still prevent a permitted contact. These conditions address contact permission, not quantitative bounds on credential issuance or message use.

For a policy that can change over time, the applicable policy version and the effect of revocation on existing credentials must additionally be specified. Quota consumption and token expiry likewise require explicit state and validity conditions. The general requirement leaves room for these obligations, but the static permission conditions above do not establish them.

A message-level authorization claim therefore requires a model connecting policy evaluation, credential issuance, and the final message-acceptance event. Chapter 6 formalizes selected consequences of decision divergence under stated assumptions; it does not establish the complete justification relation above.

Research question and scope. How can authorization decision inconsistencies be identified from recorded implementation evidence and specification-side evaluation, and their protocol consequences analyzed under explicit symbolic model assumptions? For the recorded SAGA case, we compare an independently evaluated specification decision with the archived implementation-observed decision, check their evidence provenance, and reconstruct a divergence scenario for symbolic consequence analysis. This separates decision comparison, evidence-to-scenario transformation, and verification of selected

model properties. Matching the recorded inputs and tracking their provenance does not establish that the observation and symbolic witness belong to the same real execution.

3.3 Authorization Pipeline

The relevant decision is distributed across several stages. The user's rulebook determines an intended answer for I contacting R. The Provider evaluates its stored copy of that rulebook and, if the request passes its checks, returns one-time key material. The initiating and receiving agents then complete the credential exchange; the receiving agent generates a token and later validates a token presented with a message. An authorization argument must connect the same initiating identity and receiving identity across these stages, rather than infer permission solely from the presence of a valid token. The intended progression is

$$P_R \longrightarrow \mathsf{ProviderGrant} \longrightarrow \mathsf{TokenIssued} \longrightarrow \mathsf{MessageAccepted}.$$

The arrows indicate protocol stages and authorization dependencies, not a guarantee that reaching one stage causes the next to succeed.

Each boundary requires separate evidence: a Provider response does not establish token issuance, and token issuance does not establish message acceptance. Later checks and protocol steps may fail.

When $\neg\mathsf{Allow}_{\mathrm{spec}}(P_R, I)$ holds, we distinguish three levels of violation:

1. **Provider decision violation:** $\mathsf{ProviderGrant}(I, R)$ occurs despite the policy denial.

2. **Credential issuance violation:** $\mathsf{TokenIssued}(R, I, T)$ occurs despite the policy denial. Issuance alone does not establish delivery or subsequent use.

3. **End-to-end authorization violation:** $\mathsf{MessageAccepted}(R, I, T, m)$ occurs despite the policy denial. This is acceptance at the receiving agent's application boundary.

These levels classify possible observations rather than assert results. A sound pipeline must preserve the link to the applicable decision, participants, and credential across stages. Chapter 6 specifies the corresponding events; Chapter 7 evaluates implementation observations and model results separately.

3.4 Threat Model

The following threat scenario motivates the symbolic analysis. Its identity, policy, and trust assumptions are not observations of a deployed service run. We consider a normally registered initiating agent controlled by the adversary. It can choose its contact requests and participate in the protocol under its genuine identity and certificate. The receiving user's rulebook is an existing configuration: the adversary cannot change its contents or rule order, control the Provider or CA, or obtain another honest party's private keys.

For the divergence scenario, we assume a common fixed rulebook for the Provider and receiving agent. In the symbolic model, their enforcement points consume the supplied implementation-side decision; they do not execute that rulebook or the Python matcher. The function-level observation does not establish that both deployed consumers used the same policy or consumed the recorded decision. Cryptographic checks are assumed effective. We analyze possible consequences of a decision inconsistent with the intended policy, without compromise of either enforcement point. The prescribed initiating-peer process represents the modeled interaction, not arbitrary adversary-controlled application behavior.

The adversary's objective is an execution in which the intended policy denies contact by I, yet the system performs one of the authorization-relevant actions above. At the message level, the violation condition is

$$\mathsf{MessageAccepted}(R, I, T, m) \wedge \neg \mathsf{Allow}_{\mathrm{spec}}(P_R, I).$$

This end-to-end violation is distinct from an erroneous Provider grant or credential issuance. Claiming it requires evidence that the intervening checks and protocol steps reach the receiving agent's acceptance boundary. The results in this paper concern reachability within the specified symbolic scenario. They do not demonstrate a deployed attack or a real service execution connecting the matcher observation to message acceptance.

We study this objective under a static policy. Policy updates and revocation, disagreement between stored policy versions, quantitative exhaustion of budgets or quotas, compromise of trusted parties, and arbitrary actions after contact require additional assumptions and are outside this initial question. The distinction between zero, negative, and positive configured budgets remains part of the contact-permission semantics.

4 Authorization Inconsistencies in SAGA

This chapter examines how the studied reference implementation interprets contact policy at its enforcement points. We distinguish the documented selection of a rule from the interpretation of the selected budget. The analysis concerns the fixed-policy setting of Chapter 3 and is based on source inspection; runtime propagation to the three security boundaries requires separate execution evidence. The findings concern the inspected historical revision, not the current upstream status.

4.1 Intended Contact-Policy Semantics

A contact rulebook is represented as an ordered list of rules, each carrying a `pattern` and an integer `budget`. A pattern matches an agent identifier, or AID. For contact from I to R, the relevant object is R's rulebook evaluated against I's AID: permission has a direction. Its evaluation does not distinguish individual application commands or tool actions. Throughout this chapter, *policy budget* denotes the `budget` field of a contact rule, not the communication quota associated with an access-control token.

In `contact_policy.py`, `check_rulebook()` checks rule fields and admits integer budgets from -1 upwards. The `match()` function first checks AID syntax and then tests patterns with `fnmatch.fnmatch()`. Its documentation states that the returned budget belongs to the most specific matching pattern. The function `aid_specificity()` supplies the comparison score, using weighted contributions from literals and wildcard constructs in the user and agent-name components. We use this numerical ordering as the documented selection criterion, without assuming that it is a general ordering by containment between pattern languages.

For a well-formed fixed rulebook with a unique highest-scoring match, let b^* denote the budget attached to that rule. For no match, take $b^* = 0$, consistent with the matcher's default result. The policy baseline is

$$\mathsf{Allow}_{\mathrm{spec}}(P_R, I) \iff b^* > 0.$$

The positivity convention comes from the Provider's `/access` handler, which rejects both negative and zero results. Thus the baseline combines a documented rule-selection contract with the Provider's explicit interpretation of the result. It does not assert a separate published policy language or resolve conflicts between equally specific rules. A malformed AID produces a separate error result and is outside the valid-identity cases considered here.

A positive policy decision is only permission to proceed through this check. The Provider additionally uses resource availability and remaining contact allowance, and later protocol stages impose their own checks. Accordingly, this baseline specifies contact eligibility rather than sufficient conditions for token issuance or application-message acceptance.

4.2 Rule-Specificity Selection Flaw

The implementation of `match()` does not maintain the state needed by its documented selection rule. It initializes `best_pattern` to `None` and the return budget to zero. For each matching rule, it compares that rule's specificity with `aid_specificity(best_pattern)` and updates the budget when the comparison succeeds. It never updates `best_pattern`. Consequently, later rules are compared with the initial sentinel, not with the most specific rule already encountered.

This is more precise than saying that specificity is ignored: the comparison is executed, but its reference value remains wrong. The score of `None` is -1, whereas the scores computed for well-formed patterns are nonnegative, including zero for the universal wildcard. Therefore every matching rule handled normally by the scorer passes the comparison. For a valid AID and a well-formed rule list, the returned budget is the budget of the last matching rule encountered; with no match it remains zero.

The order involved is the rulebook's list order, not the order of keys within each rule dictionary. The registration path carries the list in the `contact_rulebook` field, the Provider stores that field, and the Agent loads it into its local state. None of these paths sorts the rules by specificity. The matcher's ordinary list traversal consequently preserves the distinction between earlier and later rules.

Consider two rules that both match the same valid identity I. Let r_s have a strictly higher specificity score and budget -1, and let r_g have a lower score and budget 10. This is an abstract policy example; it does not assume that I can edit R's configuration. The documented selection is r_s in either ordering. Inspection of the loop yields the following different implementation results:

Rule order	Intended budget	Returned budget
$[r_s, r_g]$	-1 (deny)	10
$[r_g, r_s]$	-1 (deny)	-1

To compare Boolean decisions, define

$$\mathsf{Allow}_{\mathrm{impl}}(P_R, I) \iff \texttt{match}(P_R, I) > 0.$$

This is the actual matcher result interpreted by the Provider's policy check, not the success of the entire endpoint. For the first ordering above,

$$\mathsf{Allow}_{\mathrm{impl}}(P_R, I) \neq \mathsf{Allow}_{\mathrm{spec}}(P_R, I).$$

The same logical rule set thus has an order-dependent implementation result despite an unchanged unique maximum under the documented semantics. Equal scores are unnecessary for this disagreement. The observation establishes a policy-evaluation mismatch; it does not, by itself, establish any of the credential or message events defined in Chapter 3.

4.3 Budget-Semantics Inconsistency

The two enforcement points also differ in how they interpret an identical matcher result. In `provider.py`, `access()` returns a denial for `budget < 0` and separately for `budget == 0`. Only a positive value proceeds to the resource-allocation part of the handler. Writing ProviderAllow for this policy check alone gives

$$\mathsf{ProviderAllow}(b) \iff b > 0.$$

This predicate excludes the endpoint's identity, registration, and resource checks; a true value is not yet a Provider grant.

In `agent.py`, the incoming connection handler evaluates the local rulebook against the initiating AID using `match()`. It rejects only when the returned value is negative. The budget is not tested for equality to zero at this point. For this particular precheck,

$$\mathsf{AgentPrecheckAllow}(b) \iff b \geq 0.$$

Here "allow" means only that this conditional does not reject the connection. It is neither a statement that a token is generated nor a statement that an application message is accepted.

Policy result	Provider policy check	Agent precheck
$b < 0$	Reject	Reject
$b = 0$	Reject	Continue
$b > 0$	Continue	Continue

The difference includes an explicit zero-budget match and the zero returned when no rule matches. At either source of zero, the Provider treats the result as insufficient permission, while the Agent's precheck does not stop the connection. However, the ordinary fresh-contact path obtains material through the Provider first. Its zero-budget denial remains effective at

that boundary. The receiver-side difference therefore does not independently establish a complete unauthorized execution.

Several distinct quantities must be kept separate when assessing this behavior. The configured rule budget is an input to policy evaluation. The Provider's `counter` stores a remaining allowance for each initiating AID within a target Agent's record. A first allocation initializes the entry to the selected budget minus one; subsequent allocations decrement that entry, with a floor of zero. An exhausted entry blocks further allocation. Despite its name, this is remaining contact allowance, not an accumulated count of accepted application messages. Allocation consumes allowance before the later credential exchange has necessarily completed.

Token-use state is maintained separately. The `generate_token()` function sets the token's `communication_quota` from the configured `Q_MAX` value, not from the matched contact-rule budget. Token validation checks the receiver's active-token state, expiration, remaining token quota, and recipient access-control key. The conversation loop decrements token quota and also has a separate `MAX_QUERIES` guard. These checks concern token use and local conversation processing. Neither their presence nor their success establishes that the correct policy rule was selected, and the zero-budget comparison cannot be interpreted as a token-quota exhaustion test.

4.4 Cross-Layer Authorization Inconsistency

The findings concern two different relationships. The rule-selection flaw is a *specification-to-implementation inconsistency*: the matcher can return a budget that disagrees with the documented most-specific-rule contract. The zero-budget difference is an *enforcement-to-enforcement inconsistency*: two consumers of the same policy result apply different predicates. In the latter case,

$$\mathsf{ProviderAllow}(0) \neq \mathsf{AgentPrecheckAllow}(0).$$

The second disagreement does not depend on overlapping rules. Conversely, the first can change the sign of the returned budget without involving zero. They should not be assigned the same consequence merely because both concern authorization.

The Provider and receiving Agent both invoke the common matcher. The Agent's incoming policy check therefore supplies an additional enforcement point, but it does not independently recover the documented rule selection. With the same ordered rulebook and identity, both invocations inherit the same selection error. Agreement between two implementations of a check

is insufficient when both rely on the same incorrect decision procedure. At zero, even their interpretation of that shared result differs.

These observations concern the preservation of policy meaning along the pipeline defined in Chapter 3. The Provider's successful response supplies contact material, including one public one-time key and its signature, the target Agent's certificate and public access-control material, and the target user's certificate. It removes the rulebook and counter from that response. This response is not an access-control token; the receiving Agent generates the token at a later boundary.

The incoming handler performs its policy precheck before credential handling. Subsequent identity, Provider-stamp, signature, and one-time-key checks can still stop a new credential exchange. Their purpose differs from choosing the applicable contact rule. In particular, the Provider stamp authenticates an Agent card constructed at registration, rather than a decision selecting a rule for this contact. Later, `token_is_valid()` checks credential state without re-evaluating rule specificity. Thus successful cryptographic checks cannot be substituted for the missing semantic agreement.

Cryptographic correctness does not repair an incorrect authorization decision. Even assuming that signatures, certificates, token checks, and the authenticated transport work as intended, those mechanisms do not establish that a credential was made available in accordance with the user's policy. This is a separation of obligations, not a claim that every policy mismatch survives all later checks or that the entire protocol has been exercised.

4.5 Security Consequences

For the rule-selection flaw, source inspection directly establishes that a documented denial can become a positive implementation decision. Because `access()` uses that value for its policy check, the mismatch changes whether execution proceeds towards allocation. The successful allocation branch returns contact material. Conditional on the endpoint's other checks and ordinary resource requirements succeeding, the mismatch can therefore reach the Provider-grant boundary. This is a conditional control-flow conclusion, not a report of an experimentally observed grant.

In the terminology of Chapter 3, that boundary would constitute a Level 1 violation when

$$\neg \mathsf{Allow}_{\mathrm{spec}}(P_R, I) \wedge \mathsf{ProviderGrant}(I, R).$$

An incorrect positive matcher result alone is not this event: the material

must actually be released. The distinction matters because a policy mismatch can coexist with an unsuccessful endpoint request.

A Level 2 consequence would additionally require

$$\neg \mathsf{Allow}_{\mathrm{spec}}(P_R, I) \wedge \mathsf{TokenIssued}(R, I, T).$$

The shared matcher means that the receiver's policy precheck need not correct the earlier decision, making propagation relevant to investigate. Nevertheless, token generation occurs after additional checks and depends on valid protocol state. The static mismatch neither satisfies those checks by itself nor records a token being released. It is therefore insufficient as standalone evidence of credential issuance.

Likewise, the Level 3 condition

$$\neg \mathsf{Allow}_{\mathrm{spec}}(P_R, I) \wedge \mathsf{MessageAccepted}(R, I, T, m)$$

requires the receiving application boundary to be reached with an accepted message and valid credential state. Entering a credential-handling branch, generating a token, and accepting a protected message are distinct observations. This chapter establishes no execution exhibiting this final condition.

For the zero-budget discrepancy, the supported conclusion is narrower. The Agent precheck disagrees with the baseline denial at zero, but the Provider's own decision is unchanged: it rejects that value. The discrepancy alone therefore does not produce a Provider grant on the fresh-contact path. Neither token issuance nor message acceptance follows from the receiver's failure to reject at this single check. Without additional execution evidence, this finding is a latent authorization inconsistency, not an independently confirmed access violation.

The two findings thus establish concrete policy-evaluation and enforcement disagreements, with different implications for the intermediate boundaries. They show why the authorization-soundness obligations of Chapter 3 cannot be assumed from the implementation's cryptographic checks. An implementation mismatch is not automatically an end-to-end exploit. Chapter 5 examines matcher-level evidence separately from the modeled consequences of an assumed authorization divergence at material release, token issuance, and message acceptance.

5 Formal Consequences of Authorization Inconsistency

5.1 Overview

The starting point of this case study is an authorization decision inconsistent with the receiving user's policy. Following the fixed-policy scope of Chapter 3, the initiating agent has a genuine identity, and the policy is an existing configuration outside its control. The relevant evidence connects two distinct levels: a confirmed matcher-level decision mismatch and a formal analysis of its possible communication consequences. This chapter explains the historical preset-divergence model and witness; Chapters 6 and 7 present the subsequent bridge-derived execution. Both use symbolic protocol abstractions rather than executions of the complete service.

The conceptual chain is

Policy specification
↓
Matcher evaluation
↓
Authorization decision
↓
Credential issuance
↓
Message acceptance.

These arrows identify dependencies to examine, not implications that every decision necessarily reaches the next stage. In particular, a matcher result, a Provider grant, and an accepted application message require different evidence.

Cryptographic checks remain part of the modeled protocol: the consequence analysis does not disable them or assume that their primitives fail. The saved baseline diagnostic separately preserves its authentication and token-secrecy results. Those results do not automatically transfer to every query in the consequence model. The path studied here concerns the use of an incorrect authorization decision within the modeled protocol.

5.2 Matcher-Level Counterexample

The existing `proofs/test_matcher_bug.py` illustrates the rule-selection inconsistency using this ordered rulebook:

Pattern	Budget	Role
`mallory@example.com:*`	-1	Specific deny
`*`	10	General allow

Both rules match the example's initiating identity. Under the documented most-specific-rule semantics, the first rule determines the result. The expected budget is therefore -1, interpreted as denial. The illustrated implementation instead returns 10, interpreted as permission. Using the decision predicates of Chapters 3 and 4,

$$\begin{aligned} \mathsf{Allow}_{\mathrm{spec}}(P_R, I) &= \mathsf{false}, \\ \mathsf{Allow}_{\mathrm{impl}}(P_R, I) &= \mathsf{true}. \end{aligned}$$

The script is a self-contained comparison of defective and corrected selection logic with a simplified specificity score. It does not import the production matcher. Its role is to expose the decision discrepancy, not to establish general correctness of the corrected procedure.

Direct implementation evidence is provided separately by the archived `policy-matcher-sanity.json`.[2] That record uses an Alice identity with the same specific-deny/general-allow structure. It reports a valid rulebook, specificity scores of 70 and 0, an expected budget of -1, and an actual result of 10. Reversing the rule order returns -1. The identity strings in this record differ from the Mallory illustration and are not presented as the same execution.

Together, these materials establish an implementation-level authorization decision inconsistency: the policy enforcement implementation produces an authorization decision inconsistent with the intended semantics. The direct-call record does not exercise Provider requests, token issuance, or message acceptance. The existing rule order is a condition of the example, not an adversarial ability to modify the receiving user's policy.

5.3 Formalizing the Authorization Divergence

The historical model `agent_communication_authz_chat_turepass.pv`, under `proofs/proverif/`, archived in package D, asks a conditional question: given an authorization divergence, can the modeled protocol reach communication acceptance? It does not execute Python, calculate pattern specificity, or select a rule from a concrete rulebook. Instead, initialization supplies the fixed table facts `ScenarioSpecDeny` and `ScenarioImplAllow` for the scenario identifier `BuggyPolicy` and the initiating agent.

[2] The file is stored under `proofs/evidence/authz-stage1-20260916T123834944479Z/`.

This is the abstraction boundary. The terms *SpecDecision* and *ImplDecision* are analysis-level labels rather than formal event predicates. They describe the two sides of the authorization inconsistency being modeled. ProVerif models the protocol consequence after the assumed mismatch; it does not compute the matcher result. The two scenario predicates are table facts supplied at initialization. The event `SpecIntendedDeny` records the assumed policy denial. The table fact representing implementation permission allows the modeled enforcement processes to continue.

The participant directions are fixed: `PeerA` represents the receiver R, and `PeerB` represents the initiator I. The relevant observations have the following meanings:

Model observation	Meaning
`SpecIntendedDeny`(I, R)	Records the intended denial of contact by I with R.
`ProviderRelease`(R, I)	Records the release of contact material.
`TokenIssue`(R, I, t)	Records issuance of the wire token t.
`ChatAccept`(R, I, t, m)	Records R accepting message m from I using t.

The denial event uses the opposite argument order from the release and communication events. This distinction matters when relating the observations to Chapter 3's ProviderGrant, TokenIssued, and MessageAccepted concepts. That conceptual correspondence is not an implementation-to-model refinement proof.

The model includes a `ProviderReleased` table to represent the handoff from Provider release to receiver processing. Message acceptance uses an `ActiveToken` association between the receiver, sender, wire token, and sender's access-control public key. The private channel `tls_chat` abstracts authenticated, confidential transport; it is not a verification of TLS. These assumptions delimit the consequence analysis.

5.4 Formal Reachability Witness

The historical `proofs/evidence/turetrace` contains an existential symbolic reachability witness for `ChatAccept`. The projection below retains the observations relevant to authorization. Line numbers refer to that file, not to the process-position numbers printed inside the trace.

Line	Observation
14, 16	Specification-deny and implementation-allow table facts are inserted.
122	`SpecIntendedDeny` records the intended denial.
128	`ProviderRelease` records material release.
168	`TokenIssue` records the token later used in acceptance.
196	`TokenReceive` records receipt of that wire token.
202	`ChatSend` records the message with that token.
208	`ChatAccept` records acceptance of that message.

In the reachability witness, `TokenIssue` and `ChatAccept` coexist with the assumed policy denial and abstract implementation permission. Denial is a recorded scenario fact, not a blocking guard. The token recorded at issuance also appears in receipt, sending, and message acceptance; the witness therefore extends beyond credential receipt.

`ChatAccept` means that the receiving agent accepts an application message under a token-mediated communication path, after the checks represented in this model. It does not mean that a production service handled the message or that a downstream tool operation completed. It is also distinct from `Accept`, which denotes token receipt and verification in these models.

The saved result states that `not event(ChatAccept(...))` is false, with a reconstructed witness: message acceptance is existentially reachable. Together with the denial assumption, this supports the abstract authorization consequence, not a failed cryptographic authentication query. The fresh ProVerif 2.05 run reproduces this result and archives the full output.

The witness contains replicated processes and shared token state: the issuing instance at line 168 differs from the accepting instance at line 208. Fresh tokens therefore do not establish session isolation or injective agreement. The historical file omitted the final issuance-correspondence result; the fresh run records `ChatAccept` implying `TokenIssue` as true. This establishes prior issuance with the same receiver, sender, and token within the model, not intended policy permission. No independent `TokenIssue` reachability query exists; the summary distinguishes this absence from issuance observed in the witness.

5.5 Evidence Boundary

Established results. The example script and direct sanity JSON record support the matcher inconsistency. Separately, the consequence model and its witnesses support coexistence of abstract divergence, issuance, and communication acceptance. The fresh run records message acceptance as reach-

able and the issuance correspondence as true. Matcher evidence remains distinct from formal execution evidence.

Claims outside the evidence. These results do not establish a cryptographic primitive failure, certificate forgery, private-key compromise, or arbitrary attacker control. In particular, the initiating participant in the symbolic witness executes the prescribed peer process; the trace does not demonstrate arbitrary application behavior by an external attacker. The static policy and the genuine-identity setting remain those of Chapter 3. The zero-budget discrepancy discussed in Chapter 4 is not an additional completed path in this chapter.

Remaining limitation. The historical model D abstracts the matcher result using preset facts. For the recorded evaluation, bridge v1.1.1 now independently derives the specification-side decision, binds the archived implementation observation, checks their common evaluation context, and reconstructs the scenario region used in a separate derived-model execution (package B). Chapters 6 and 7 describe this checked evidence transfer and its results. ProVerif still does not execute Python. Symbolic role binding remains an abstraction; general implementation-to-model refinement and correspondence with deployed consumers remain unproved. The `BuggyPolicy` identifier is a scenario-local label, not an encoding of the concrete rulebook.

The historical consequence trace lacks an independent run manifest for its original execution. Historical package D provides reproducible provenance for an independent execution of the unchanged ProVerif consequence model.[3] It records the execution timestamps, Git commit and branch, verifier version, executable and model hashes, command, exit status, raw standard output and error, and a summary extracted from that output. This package documents the new run; it does not retrospectively establish the origin of the historical `turetrace` file.

Package D did not supply a decision bridge. The later package B separately records checked decision-value transfer, scenario reconstruction, and a fresh ProVerif execution with the same two queries.[4] B does not retroactively establish the provenance of D or `turetrace`. Neither package establishes deployed-service reproduction or general refinement. The recorded

[3]Historical package D: `proofs/evidence/authz-turepass-20260919T090826295518Z/`.

[4]Bridge-derived package B: `proofs/evidence/authz-bridge-turepass-20260922T111657792091Z/`.

policy evaluation and symbolic witness are not evidence of the same concrete service session.

6 Formalization

6.1 Modeling Approach

The formal model analyzes conditional consequences of authorization decision divergence for the recorded SAGA case. An archived implementation observation and independent specification-side evaluation yield a checked decision pair. An evidence-backed decision transformation procedure reconstructs a fixed symbolic scenario from that pair, which ProVerif then analyzes under explicit protocol assumptions (Figure 1). The decision bridge is a component of this analysis procedure, not a formal model translation framework. It does not construct the protocol abstraction or establish refinement between Python and the symbolic model. ProVerif neither executes Python nor discovers the implementation inconsistency. This analysis retains Chapter 3's fixed-policy setting and studies selected consequences; it does not prove the ideal decision-to-access justification relation.

6.2 Protocol Model

The Provider abstracts registration checks, participant authentication, and the authorization-related release of contact material. `PeerA`, the receiver R, issues tokens and receives application messages. `PeerB`, the initiator I, requests a token and sends a message after receiving it. A CA process supplies certificates. Replicated processes use symbolic cryptography and public communication; the private `tls_chat` channel abstracts authenticated, confidential transport.

We distinguish the divergence model, `turepass`, from three independent gate controls. The controls differ only in their preset gate permissions. They use a fixed private token, whereas `turepass` generates fresh tokens. The controls are not verified repairs or a refinement of that model. The current bridge-derived model preserves the consequence structure of `turepass`, with scenario input reconstructed before verification; the historical and bridge-derived executions have separate evidence packages.

6.3 Evidence-Backed Decision Transformation Procedure

Recorded input and provenance. Bridge v1.1.1 checks an ordered policy P, its matched initiating identity I, a policy hash, the historical matcher revision and source hash, and the archived observation's location and hash. These fields track the recorded inputs and evidence used in the comparison; they do not identify a real authorization event or protocol session. The matched

identity is the initiator, not the receiver. Policy labels and rule identifiers organize the artifact; they do not establish deployed policy ownership.

Independent specification decision. An executable evaluator supports the finite pattern subset needed for the recorded case: `*`, lowercase ASCII literal AIDs, and literal UIDs followed by `:*`. It independently computes matching and the numerical specificity described in Chapter 4, then selects the unique highest-scoring match. It neither imports production `match()` nor reads the archive's expected result as an answer. Budgets are classified as Negative, Zero, or Positive; only Positive grants contact permission. No match yields Zero. Tied highest scores and unsupported patterns produce `Unsupported`; malformed inputs produce `Invalid`. Neither status is silently interpreted as denial. This is finite executable specification semantics, not theorem-proved evaluator correctness or complete `fnmatch` semantics.

Implementation observation. The archived matcher record contains a direct function-level call to the unmodified matcher implementation for `alice@example.com:agent`. A specific denial followed by general permission returns 10; the reversed order returns -1. The bridge loads the observation for the selected order, checks its identity and ordered rules, and binds its source hash to the historical Git revision using explicit line-ending encodings. Equality with the current checkout is not required. The observed implementation decision is an archived execution result, not a prediction by the evaluator or an execution inside ProVerif.

Decision-pair construction. For these checked recorded inputs, let $D_{\text{spec}}(P, I)$ be the independently computed specification decision and $D^{\text{obs}}_{\text{impl}}(P, I)$ the archived matcher result interpreted using Chapter 3's positive-budget convention. The evaluator selects the specific rule, with score 70 rather than 0 and budget -1; the bound observation supplies budget 10. Thus,

$$(D_{\text{spec}}(P, I), D^{\text{obs}}_{\text{impl}}(P, I)) = (\mathsf{Deny}, \mathsf{Allow}).$$

The procedure compares the decisions only after validating the recorded ordered rules, matched identity, and observation provenance. This establishes an evidence-linked comparison, not equivalence of real executions. The observation-based relation does not define implementation outputs for arbitrary inputs.

Scenario reconstruction. Before generating the full model, the bridge recomputes the specification decision, reloads and validates the observation, and rechecks their context; saved decision labels are not trusted. It locates a unique scenario region in the hash-pinned consequence template and joins the retained prefix and suffix around the generated statements in Table 1.

The executed model reconstructs its authorization-divergence scenario region from the checked pair, while protocol processes, events, channels, token logic, and queries remain frozen. Only the provenance header, scenario region, and three approved comment-only normalizations may differ. Exact structural checks and mutation tests support this artifact-level checked transformation, not semantic refinement.

Automation and analysis choices. Within the supported finite input grammar, the procedure automates specification-side decision evaluation, observation validation, provenance checks, and decision-pair comparison. Scenario generation is restricted to the checked Deny/Allow pair and the fixed consequence template; other decision pairs do not generate a full model through this adapter. Policy abstraction, including the supported grammar and budget interpretation, remains an analysis choice. Symbolic role mapping, protocol model construction, and query design are also specified by the analyst rather than derived from the observation. The independent gate controls use analyst-selected Provider/receiver permission combinations; they are not additional implementation-observed decision pairs.

Evidence or analysis statement	Bridge-derived model representation
Independently evaluated budget -1: deny	`ScenarioSpecDeny(BuggyPolicy,aid_B)` table fact.
Recorded budget 10: allow	`ScenarioImplAllow(BuggyPolicy,aid_B)` table fact.
Authorization divergence	Coexistence of these facts; no separate `SpecDivergence` event is declared.
Modeled message acceptance; not a matcher observation	`ChatAccept`(R, I, t, m) in the symbolic witness.

Table 1: Checked decision values and generated scenario facts. Symbolic participant bindings remain modeling abstractions.

Remaining abstraction boundary. The concrete initiating identity is represented by `aid_B`; the unobserved receiving policy owner by `aid_A`; and the policy evaluation by the scenario-local label `BuggyPolicy`. These role bindings remain analysis abstractions. The matcher archive identifies neither a deployed receiver nor a deployed policy owner, and does not describe the same concrete service session as the symbolic witness. Both Provider and receiver consume the fixed allow fact; their handoff and behavior are not validated by the matcher observation. Avoiding per-run manual transcription within this fixed adapter therefore does not remove the implementation-to-model refinement boundary.

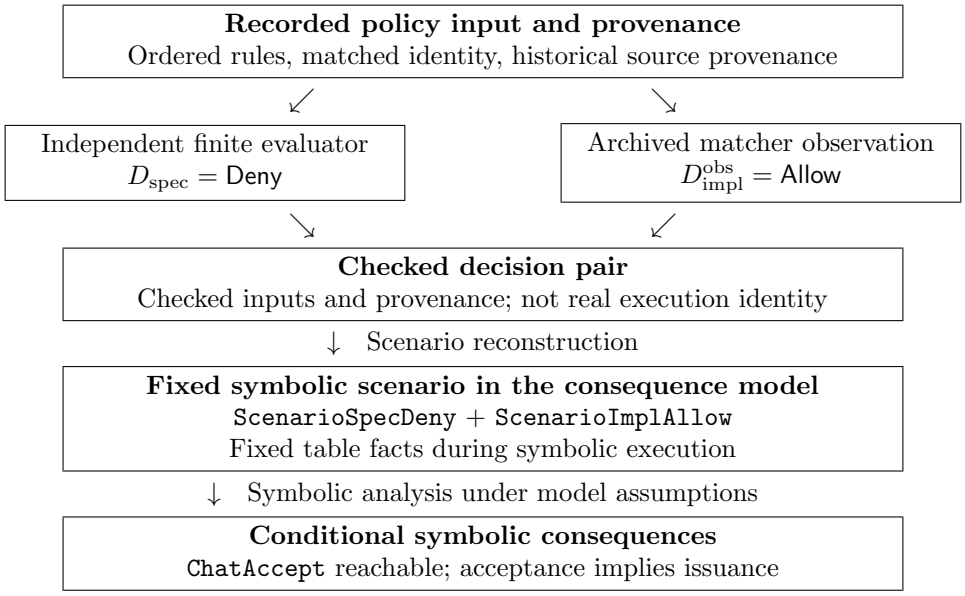


Figure 1: Evidence-backed decision transformation for the recorded SAGA case. The bridge checks and transfers decision values into a fixed scenario; policy abstraction, role mapping, protocol construction, and query design remain analysis choices. The arrows do not establish refinement or equivalence with a real service execution.

6.4 Authorization Events and State

In both the historical and bridge-derived models, `ScenarioSpecDeny` and `ScenarioImplAllow` are initialized table facts, not Python matcher events or ProVerif-computed policy decisions. The facts remain fixed symbolic inputs during ProVerif execution. In the bridge-derived run only, their decision values are reconstructed externally from the checked pair. The historical `turepass` execution used a manually stated divergence scenario. These separate evidence packages must not be conflated. The facts share a scenario identifier and initiating identity. `SpecIntendedDeny`(I, R) records the specification-side denial represented in the symbolic scenario. The following protocol events use receiver-first arguments; t denotes the wire token.

Event	Observation
`ProviderRelease`(R, I)	Contact material released.
`TokenIssue`(R, I, t)	Token issued for the initiator.
`TokenReceive`(R, I, t)	Initiator received and verified the token.
`ChatSend`(R, I, t, m)	Initiator sent message m with t.
`ChatAccept`(R, I, t, m)	Receiver accepted the application message.

The `ProviderReleased` table abstracts a handoff to receiver processing. `ActiveToken` associates receiver, initiator, token, and the sender's access-control public key. Gate controls additionally require `ProviderAuthorized` for release and `ReceiverAuthorized` for issuance. Receiver processing also requires the handoff state. These tables govern execution; recording an event alone does not grant or deny permission.

Table 2 separates scenario assumptions from observations and verification queries. Divergence is an input to the consequence model; its implementation evidence is independent of the solver results. Gate effects require comparisons between scenarios, not merely the declaration of a release or issuance event.

6.5 Verification Queries

Reachability. Both divergence models query whether `ChatAccept` can occur. The controls separately query `ProviderRelease`, `TokenIssue`, and `ChatAccept` for the fixed peer identities. A false `not event(...)` result with a reconstructed witness establishes an existential modeled consequence under the scenario assumptions, not a security guarantee. There is no inde-

Claim or assumption	Formal representation and scope
Authorization divergence input	`ScenarioSpecDeny` and `ScenarioImplAllow` table facts; externally reconstructed from the checked pair in the bridge-derived model, not computed by ProVerif.
Provider permission affects release	`ProviderRelease` reachability across gate controls.
Token issuance depends on gate state	`TokenIssue` reachability across gate controls.
Acceptance requires prior issuance	`ChatAccept` ⇒ `TokenIssue`; queried in the divergence models and all three controls.
Acceptance has a matching send	`ChatAccept` ⇒ `ChatSend`; queried in the three gate controls only.

Table 2: Claim-to-Formalization Mapping. Correspondence arguments and their limitations are specified below.

pendent `TokenIssue` reachability query in either historical `turepass` or its bridge-derived execution.

Correspondence. Both divergence models and the three controls query

$$\mathsf{ChatAccept}(R, I, t, m) \Longrightarrow \mathsf{TokenIssue}(R, I, t).$$

The controls also query

$$\mathsf{ChatAccept}(R, I, t, m) \Longrightarrow \mathsf{ChatSend}(R, I, t, m).$$

Variables are universally quantified. These non-injective correspondences require prior issuance for the same participants and token, or prior sending of the same message, respectively. They do not establish policy permission, unique sessions, or Chapter 3's full decision-to-access justification.

Gate placement. The controls preset Provider/receiver permissions to allow/allow, deny/allow, and allow/deny. Comparing their three reachability queries locates blocking stages under the shared handoff abstraction.

6.6 Formal Results

The historical `turepass` archive reports message acceptance reachable under its preset divergence and the acceptance-to-issuance correspondence true. Separately, a fresh ProVerif 2.05 execution of the bridge v1.1.1 derived model reports the same two query outcomes after scenario reconstruction: `ChatAccept` is reachable, and acceptance implies prior `TokenIssue`. The

new archive binds the input decision pair, generated model, clean code revision, and verifier output. It does not retroactively upgrade the historical execution or constitute a production-service run. Token issuance occurs in the witness; neither divergence model has a standalone issuance-reachability query.

For the controls, the results are:

Provider / Receiver	Release	Issue	Accept
Allow / Allow	Reachable	Reachable	Reachable
Deny / Allow	Unreachable	Unreachable	Unreachable
Allow / Deny	Reachable	Unreachable	Unreachable

Provider denial stops release and downstream events. Receiver denial stops issuance and message acceptance after material release remains possible. Both control correspondences are true in all three scenarios. They are non-vacuous on the allow/allow path; in each denial scenario, acceptance is unreachable and the correspondences hold vacuously.

These results are documented in archived summaries and raw output. The historical divergence package D records its command, version, input hash, and zero exit status for an independent historical run, separate from the earlier `turetrace` excerpt.[5] The bridge-derived execution is recorded separately in `authz-bridge-turepass-20260922T111657792091Z/` under the same evidence directory. Its archived results support the current scenario reconstruction, while gate results retain their original scope.

6.7 Formalization Boundary

The matcher inconsistency is demonstrated at implementation level, and ProVerif models its protocol-level consequence under explicit assumptions. The external evaluator computes finite specification decisions, and the bridge checks their connection to archived implementation observations and generated scenario input. These executable checks are not proofs of evaluator correctness, complete Python or `fnmatch` semantics, or arbitrary policy correctness. ProVerif establishes reachability, blocking, and correspondences only within the stated symbolic scenarios; it does not prove Python execution correctness or a production deployment result. In particular, provenance tracking does not prove Python matcher correctness, complete SAGA

[5] Evidence under `proofs/evidence/`: `authz-turepass-20260919T090826295518Z/` and `authz-chat-gates-20260918T023621017067Z/`.

authorization correctness, or equivalence between the recorded function execution and the symbolic witness. Chapter 3's full justification relation remains an ideal objective rather than a verified property here.

The prescribed peer processes do not represent arbitrary application behavior under attacker control. Private transport and shared state remain abstractions; token expiry, quota consumption, revocation, and policy updates are not covered. No cryptographic primitive failure or key compromise is established. The recorded correspondences are not a revalidation of every original authentication or secrecy property. For the fixed Deny/Allow adapter, per-run decision transfer is automated, but symbolic role binding, correspondence with deployed consumers, general implementation-to-model refinement, and service-level reproduction remain outside the completed evidence. The bridge implements an evidence-backed decision transformation procedure for this case, not a verified compiler or a new general verification framework.

7 Evaluation

7.1 Evaluation Questions

We evaluate evidence-backed authorization decision analysis for the recorded SAGA case through four questions using archived evidence:

Q1. Does the recorded matcher decision agree with the documented most-specific policy semantics?

Q2. Do the original authentication and secrecy queries establish the authorization relation required by this case study?

Q3. Which symbolic consequences are reachable in the divergence scenario? Separately, which stages are blocked by the preset Provider and receiver permissions in the independent gate-control models?

Q4. Can the analysis procedure check and transfer the specification-side decision and implementation-observed decision into the fixed Deny/Allow scenario, tracking provenance without per-run manual transcription?

Q4 concerns evidence transfer and artifact integrity, not a new security theorem. Implementation observations, bridge checks, and symbolic results retain distinct evidentiary roles. This evaluation reports already archived executions; no new experiment is performed for this paper revision. As an empirical complement, Phase 1 consumer-side propagation validation checks whether fresh matcher observations are linked through the production consumers to token issuance and storage under controlled conditions. This experiment is separate from both decision transfer into the model and symbolic message-acceptance results.

7.2 Experimental Setup

The matcher record uses Python 3.14.3 for direct function calls. The formal records use ProVerif 2.05 on Windows. Four historical evidence packages provide the matcher and formal-analysis inputs and results: Stage 1 (S), a historical baseline scenario with preset divergence (D), a bridge-derived decision divergence scenario (B), and independent gate-control analysis (G). [6] Their manifests record execution metadata, source revisions, commands,

[6] Packages under `proofs/evidence/`: S = `authz-stage1-20260916T123834944479Z/`; D = `authz-turepass-20260919T090826295518Z/`; B = `authz-bridge-turepass-20260922T111657792091Z/`; G = `authz-chat-gates-20260918T023621017067Z/`.

input hashes, and output provenance; raw standard output, standard error, and verification summaries accompany the runs. The reported model inputs match their recorded hashes. These archives originate from separate revisions, rather than a single integrated service experiment. D executes a preset divergence scenario; B executes a scenario reconstructed from a checked decision pair by bridge v1.1.1. B supplies separate execution provenance and does not retroactively upgrade D. The separate Phase 1 consumer archive uses Python 3.12.7 and MongoDB 8.0.17 on Windows; its setup and results appear in Section 7.5. It is not another ProVerif run.

These packages address distinct analysis tasks. S supplies the matcher observation and verification-coverage diagnostic. B connects the observed decision inconsistency to a fixed symbolic scenario through the checked transformation procedure. D is the historical baseline for that consequence structure, not a correct-implementation or repaired-system baseline. G studies enforcement positions using analyst-selected Provider/receiver permissions; its permission pairs are not specification/implementation decision pairs derived from matcher observations. Gate placement analysis therefore complements decision inconsistency analysis rather than extending its implementation evidence.

Table 3 reports the seven communication-model runs used here. Each completed with exit status zero. Counts refer to final query results, not to satisfied security properties: reachability is reported through negated-event queries. Durations are archived elapsed times rounded to one decimal place. B's recorded interval includes the verifier execution and subsequent tests and integrity checks; it is not a verifier-only measurement. These times do not measure service overhead or establish a performance comparison; repeated measurements and a controlled hardware benchmark are outside this evaluation.

Package	Model / scenario	Results	Seconds
S	Original communication	11	334.6
S	Authorization diagnostic	12	353.7
D	Historical baseline divergence	2	598.4
B	Bridge-derived divergence	2	810.9
G	Provider allow / Receiver allow	5	813.6
G	Provider deny / Receiver allow	5	507.1
G	Provider allow / Receiver deny	5	552.5

Table 3: Archived communication-model runs and recorded elapsed times. B includes post-verification checks.

7.3 Matcher Evaluation

The intended semantics select the most specific matching rule. The self-contained `proofs/test_matcher_bug.py` illustrates the inconsistency with a specific deny rule for `mallory@example.com:*` followed by a wildcard allow rule. This explanatory script defines its own matcher variants and simplified specificity function; it does not import the implementation matcher.

Package S supplies direct implementation evidence in `policy-matcher-sanity.json`. Its recorded invocation uses `alice@example.com:agent`, a specific rule `alice@example.com:*` with budget -1, and a wildcard rule with budget 10. The rulebook is valid, and the recorded specificity scores are 70 and 0. The documented semantics therefore require budget -1 in either ordering. Table 4 reports the saved results for these two orderings.

Rule order	Expected budget	Returned budget
Specific deny, then wildcard allow	-1 (deny)	10 (allow)
Wildcard allow, then specific deny	-1 (deny)	-1 (deny)

Table 4: Direct matcher observations for the recorded Alice rulebook.

The first ordering establishes a concrete policy decision mismatch; the second shows that reordering the same rules changes the result. This is an implementation-level, order-dependent authorization decision inconsistency. The record covers matcher calls, not Provider processing, token issuance, or message acceptance in a running service. Two observed orderings do not estimate the prevalence of the defect across rulebooks or deployments.

For the same ordered rulebook, B's independent finite evaluator selects `r0`, the unique highest-specificity match, with score 70 and budget -1. Its specification decision is therefore Deny; the bound implementation observation supplies budget 10, classified as Allow. The archive's historical expected-result field is not used as the evaluator's answer. This independently derived decision and the direct observation provide the two sides of the recorded inconsistency.

7.4 Evidence-Backed Decision Transformation

Package B addresses Q4 by evaluating the decision-bridge component of the analysis procedure for the recorded case. It checks the independently derived Deny decision and observed Allow decision against the recorded ordered rules, matched identity, and historical source provenance. Rechecking that pair reconstructs the `ScenarioSpecDeny` and `ScenarioImplAllow`

table facts in the full model. They remain fixed symbolic inputs during ProVerif execution, not matcher execution events. Within the fixed Deny/Allow adapter, the recorded values are transferred without per-run manual transcription. Policy abstraction, symbolic role mapping, protocol construction, and query design remain analysis choices as specified in Section 6.3.

The archived ProVerif 2.05 run of this derived model exits with status zero and reports exactly the two expected queries: `ChatAccept` is reachable, and acceptance implies prior `TokenIssue`. The manifest records bridge v1.1.1, a clean pre-run code revision, and unchanged tracked files after execution. Its hashes bind the archived observation, decision pair, template, generated scenario and full model, and verifier executable and outputs. The separate run-start record, generated input, command, exit status, and final manifest document this execution. Hashes provide integrity binding, not independent attestation of archive authenticity.

Separately, the saved test log reports 25 tests passed with exit status zero. These cover historical replay independent of current source equality; rejection of missing historical revisions and wrong blobs; explicit line-ending provenance; decision-pair tampering; template-hash mismatch; missing or duplicate scenario regions; protocol, query, channel, and token mutations; and enforcement of the approved comment changes. They also cover normal allow/deny and reversed-order controls, together with unsupported patterns, ties, and invalid inputs. These are artifact and reproducibility checks, not 25 security properties or a proof of the evaluator. Their success supports the checked transfer for this recorded case, not general implementation-to-model refinement.

7.5 Consumer-Side Propagation Validation

Purpose and setup. Phase 1 tests whether observed matcher decisions propagate through the consumer-side authorization workflow to token issuance and storage in a controlled environment. The independent harness executes the production Provider `/access` route with a real isolated MongoDB instance and passes the returned one-time key (OTK) to the production receiver handshake. Both consumers call the production matcher through observers that preserve its inputs, return values, and exceptions. The receiver re-evaluates its policy; a decision object is not transmitted between consumers. Production signature checks, OTK consumption, key derivation, token encryption, and `active_tokens` storage remain in the executed path. Registration state and certificate retrieval use fixtures; a Flask

test client and an in-memory connection replace network transport and TLS. The sole execution cutoff is `receive_conversation`, after the token has been stored and sent. Its stub records the boundary without executing conversation logic or `local_agent.run`.

Design. Each case starts with fresh cryptographic material, an empty counter and token store, and one available OTK. The initiator is `alice@example.com:agent`; the fixture receiver is `bob@example.com:receiver`. The Provider and receiver hold identical ordered policies. Let s denote the specific pattern `alice@example.com:*` and w the wildcard `*`. A/A uses $[(s, 10), (w, 10)]$; D/D uses $[(w, 10), (s, -1)]$; D/A uses $[(s, -1), (w, 10)]$. Here the first label is the independently evaluated specification decision and the second is the observed implementation decision, not the Provider/receiver permission pair used in package G. The harness reuses the bridge's finite evaluator read-only; it neither imports a historical matcher observation into the run nor changes the symbolic model. Zero budgets are excluded because the Provider rejects zero whereas the receiver's preliminary policy check rejects only negative budgets.

Spec./impl.	Provider HTTP	OTK released	Token sent/stored
A/A	200	Yes	Yes
D/D	403	No	Not reached
D/A	200	Yes	Yes

Table 5: Phase 1 consumer-side propagation results. Each case ran once; message acceptance was not executed.

Results and evidence. In D/A, the specification selects budget -1, but both consumer-local matcher calls return 10. The Provider returns HTTP 200, releases its OTK, and initializes the counter to 9. The response, receiver handshake, and initial OTK state share the same key hash; the receiver consumes that key. A single token is entered in `active_tokens` and sent, with matching token hashes across the send record, boundary observation, and stored state. Initiator-side decryption agrees with the stored token metadata, including the recipient key and initial quota. This is evidence of propagation to issuance/storage, not message acceptance. A/A confirms that the controlled setup can execute the permitted path. D/D returns HTTP 403 with unchanged state and no downstream execution; it is a Provider-blocking negative control, not an observed receiver denial. The A/A, D/D, and D/A cases pass 35, 16, and 35 assertions, respectively;

the archive also records eight passing harness tests. Assertion counts are execution checks, not additional security properties.

The successful evidence archive is `20260926T070021921746Z-d64c8064` under `proofs/consumer_validation/evidence/`. Its `manifest.json` and `run-start.json` record the run identity, environment, declared substitutions, source hashes, and execution status. Per-case JSON inputs, event logs, and results retain the policies, observations, HTTP response, state changes, and assertions; the manifest binds these artifacts by hash. The run records its dirty working-tree state and actual source-byte hashes rather than claiming a clean revision. These are fresh consumer observations, separate from S, B, and the two earlier failed harness archives.

Scope. The endpoint is token issuance and storage. No application message, LLM call, or tool execution occurs. One execution per case provides a bounded functional observation, not a reliability or performance study. Transport and registration fixtures preclude a deployed-service exploit claim. The experiment neither equates its execution with a symbolic witness nor establishes Python/ProVerif refinement or correctness for arbitrary policies.

7.6 Formal Verification Results

Authorization coverage gap. The original communication model in S produces eleven results: seven event-reachability results, three authentication correspondences, and one token-secrecy result. The diagnostic model preserves these results and adds the authorization correspondence

$$\mathsf{Accept}(R, I, t) \Longrightarrow \mathsf{PolicyAllow}(R, I).$$

This added correspondence is false, while the three authentication correspondences and token secrecy remain true. Here, `Accept` denotes receipt and verification of a token by the initiating peer; it is not application-message acceptance.

The diagnostic process never emits `PolicyAllow`; its `PolicyDeny` event records denial without guarding execution. Consequently, the failed correspondence exposes an authorization obligation absent from the original verification; it does not independently demonstrate a matcher execution error. The retained authentication and secrecy results apply to these Stage 1 queries. They are not automatically inherited by the later communication models.

Historical baseline scenario (D). Package D contains two final results for the preset `turepass` scenario. The query asserting that `ChatAccept` is unreachable is false, with a reconstructed formal reachability witness. Under preset `ScenarioSpecDeny` and `ScenarioImplAllow` facts, the witness contains intended denial, material release, token issuance, and application-message acceptance. This is a symbolic reachability consequence of the divergence abstraction.

The second result establishes

$$\mathsf{ChatAccept}(R, I, t, m) \Rightarrow \mathsf{TokenIssue}(R, I, t).$$

Thus, acceptance can coexist with the assumed policy denial while still having corresponding prior token issuance. Credential provenance and policy permission remain separate obligations. Issuance is observed in the witness; this model has no standalone `TokenIssue` reachability query and no `ChatAccept`–`ChatSend` correspondence query. The complete D archive supplies these two final results with independent execution provenance, beyond the historical `turetrace` excerpt.

Bridge-derived decision divergence scenario (B). The separate execution archived in B obtains the same two outcomes after reconstructing the scenario from the checked pair. Protocol statements and queries are intentionally preserved, so matching outcomes are expected; output agreement alone does not prove bridge correctness or execution freshness. B adds no query. As in D, token issuance occurs in the witness, but there is no standalone `TokenIssue` reachability query or acceptance-to-send query. The new evidence connects recorded decision values to the scenario used in this execution without turning the symbolic witness into a running-service observation.

Independent gate-control analysis (G). Package G evaluates three fixed pairs of Provider and receiver permissions. These inputs are preset enforcement conditions, not new observations of matcher decision divergence. The comparison isolates modeled blocking positions within G; it does not test a fix to the inconsistency recorded in S. Each control has three reachability queries and two correspondences. Table 6 translates the reachability results into reachable (R) or unreachable (U), avoiding ambiguity about negated-query truth values.

Within these gate-control models, Provider denial prevents release and downstream actions. Receiver denial prevents issuance and acceptance,

Provider	Receiver	Release	Issue	Accept
Allow	Allow	R	R	R
Deny	Allow	U	U	U
Allow	Deny	R	U	U

Table 6: Gate-control reachability: `ProviderRelease`, `TokenIssue`, and `ChatAccept`.

while Provider release remains reachable. The allow/allow control establishes a reachable permitted path in the symbolic model. Both acceptance-to-issuance and acceptance-to-send correspondences are true in all three controls. In the denial cases they hold vacuously, because acceptance is unreachable; only the allow/allow case supplies a non-vacuous control. These experiments locate blocking stages within the model, rather than evaluate a deployed repair. The controls use a fixed private token, unlike the fresh tokens in D and B, and are not verified repairs or refinements of either divergence model.

7.7 Security Findings Summary

The evidence supports four complementary findings. First, direct matcher calls demonstrate a decision inconsistent with documented specificity semantics. Second, the coverage diagnostic distinguishes authorization from the authentication and secrecy properties actually queried in Stage 1. Third, independent finite specification evaluation and a provenance-bound observation yield a checked Deny/Allow pair for the same recorded policy evaluation; the bridge reconstructs the formal scenario from that pair. Fourth, the fresh derived-model execution reaches `ChatAccept` while establishing its correspondence to prior issuance; the independent gate controls identify distinct release and issuance boundaries.

Separately, Phase 1 supplies consumer-side propagation evidence for the tested policies: fresh matcher observations are linked to production Provider and receiver execution through token issuance and storage under the declared fixtures. It does not extend the symbolic message-acceptance result into an implementation observation of message acceptance.

Together, these findings support evidence-backed analysis of the recorded SAGA authorization decision inconsistency. The implementation observation and specification-side evaluation establish the local discrepancy; the bridge component supplies checked evidence transfer into B. B characterizes a conditional symbolic consequence, while G separately characterizes mod-

eled enforcement positions. Neither the reachability results nor the gate controls establish complete authorization correctness or a verified fix. The bounded consumer experiment is neither a deployed-service exploit reproduction, a refinement proof, nor a deployment-scale security evaluation. No evidence layer substitutes for another. No result establishes cryptographic primitive failure, certificate forgery, key compromise, or arbitrary attacker control of the modeled peer processes.

7.8 Limitations

The procedure provides executable decision transfer with provenance checks for the recorded policy input, but does not prove refinement between Python and the symbolic protocol model. The external evaluator covers a finite pattern grammar; its correctness is not theorem-proved. Neither complete Python or `fnmatch` semantics nor correctness for arbitrary policies is established. ProVerif does not execute the matcher. The evaluated properties do not discharge Chapter 3's ideal decision-to-access justification relation or establish authorization correctness in real service executions.

Mapping the concrete evaluation to `aid_B`, `aid_A`, and `BuggyPolicy` remains an analysis abstraction. The historical function observation identifies neither a deployed receiver nor a deployed policy owner, and is not evidence of the same concrete service session as the symbolic witness. The bridge does not validate production consumers. Phase 1 separately observes their use of freshly computed matcher results through issuance/storage under fixtures; it establishes neither equivalence to the modeled consumers nor a shared execution with the historical record.

The model abstracts both transport and handoff through its private `tls_chat` channel and `ProviderReleased` table, respectively. The evaluated queries do not establish token expiry, quota consumption, revocation, policy-update behavior, session isolation, or injective agreement. No complete production-service reproduction or deployment-scale measurement is reported. The historical observations do not establish current upstream vulnerability status or prevalence across deployments. Future work may establish stronger semantic correspondence and evaluate service behavior and other agent systems; the present case study does not establish those outcomes or generality.

8 Discussion

8.1 Why Authentication Proofs Are Insufficient

Authentication, confidentiality, and secure channels provide complementary foundations for agent communication. Authentication addresses who is communicating; confidentiality concerns who can learn the protected information. Neither establishes whether the receiving user's policy permits that communication. Correctly identifying a participant and protecting a credential in transit do not establish the participant's entitlement to receive or use that credential.

The SAGA case study makes this distinction concrete without identifying an authentication failure. The Stage 1 results retain the three queried authentication correspondences and token secrecy, while the authorization diagnostic exposes an obligation absent from that model. Separately, the bridge-derived consequence model permits message acceptance after an externally established policy-decision inconsistency is transferred into its symbolic scenario. This does not indicate cryptographic failure or invalidate an authentication proof. These results have different scopes; the Stage 1 guarantees do not automatically transfer to all later models or to deployments.

Under the ideal authorization objective formulated in Section 3, accepted contact needs an applicable policy decision linked to the same participants and credential. This objective motivates the study; the full justification relation is not established by the evaluated properties. Prior issuance establishes credential provenance, but not permission to issue. Authentication establishes the identities to which authorization decisions must be bound; authorization determines what those identities may do.

8.2 Formal Verification Coverage

ProVerif verifies modeled properties under explicit assumptions. Authentication correspondences constrain event origins; secrecy queries constrain adversarial knowledge. An absent or abstract authorization decision layer leaves implementation authorization correctness unestablished, without invalidating the properties actually verified.

The distinction is particularly important in the diagnostic model, where `PolicyAllow` is never emitted. Its failed correspondence identifies missing authorization support in the model; it does not independently detect the Python defect. Coverage also depends on how security-critical implementation decisions are connected to their formal abstractions. For formal consequence analysis, that connection has three distinct evidentiary roles: histor-

ical function-level observation, checked decision transfer, and symbolic consequence analysis. The bridge's independent evaluator computes the finite specification-side decision. The bridge checks the corresponding recorded policy input and observation provenance, constructs the decision pair, and reconstructs the fixed scenario from that pair. The resulting model is then executed by ProVerif.

The bridge narrows the evidence gap for the recorded case by generating decision labels for the supported fixed Deny/Allow scenario from the checked pair. The specification-side Deny comes from the independent evaluator, while the implementation-side Allow comes from the archived matcher execution. ProVerif analyzes consequences of the resulting scenario table facts; it neither executes Python nor discovers the implementation inconsistency. Mechanizing this transfer does not eliminate the abstraction between concrete deployment roles and symbolic participants or prove implementation refinement. Historical package D used a preset divergence; package B reconstructs decision values while preserving the symbolic consequence structure. B does not retroactively change D's provenance.

Three questions therefore require separate justification: was the decision computed according to the intended policy; was it correctly consumed at the relevant enforcement boundary; and do authentication and credential properties hold once execution proceeds? The third cannot substitute for the first, while the gate controls show why the second depends on where enforcement occurs. This distinction organizes the case study's security argument rather than establishing a universal security theorem. Correspondences also require reachability context: they hold vacuously in the denial controls. The concern is alignment between implementation evidence, model, query, and claim, not a limitation attributed to the verification tool. The case study demonstrates a reproducible evidence chain with distinguishable observation, specification evaluation, checked scenario construction, and symbolic analysis; it does not introduce a new policy logic or verified compiler.

8.3 Policy Semantics as Security-Critical State

Authorization decisions constitute security-critical derived state: they determine whether an enforcement point may release material, issue a credential, or accept communication. Their meaning depends on the relationship between policy semantics, evaluation, and enforcement. Protecting a decision in transit or checking the resulting credential does not establish that the evaluator computed the intended permission.

In this case study, rule ordering, specificity, and budget interpretation

connect the policy to that decision. Specificity defines which matching rule should determine the result; the observed order dependence shows why list traversal must preserve that rule-selection contract. The selected budget then needs an explicit permission interpretation. In the recorded case, -1 denotes denial and 10 denotes permission. This use of a budget as a contact decision is distinct from enforcing quota consumption over subsequent messages, which the evaluated models do not establish.

Tracing the recorded input, specification evaluation, implementation observation, and reconstructed formal scenario makes the origin of an authorization state inspectable. Otherwise, a consequence analysis could verify properties of a supplied permission without establishing that the permission follows the intended policy. The bridge addresses decision-value transfer for the recorded case; correctness of the evaluator and of the downstream consumers remains a separate obligation.

The gate controls show why the decision's consumer also matters. Receiver denial blocks issuance and acceptance while Provider release remains possible; Provider denial blocks the earlier release boundary. These are results of the gate-control models, not validation of equivalent gates in a running deployment. Decision correctness and enforcement placement are distinct questions: which permission was derived, and at which boundary was it consumed? Policy meaning and the protected action must be connected at each relevant stage. These observations motivate treating policy evaluation as security-critical code. They neither establish a deployed repair nor imply that all policy engines contain the observed inconsistency.

8.4 Consumer-Side Validation Boundary

Phase 1 adds a separate execution check to the evidence-backed analysis procedure (Section 7.5). For the tested policies, fresh matcher observations are linked through the production Provider `/access` route and receiver handshake to token issuance and `active_tokens` storage. The receiver independently calls the matcher; the observed workflow is connected by the released OTK and the issued token, not by transporting a single decision object between consumers. The A/A control exercises the permitted path. D/D establishes that Provider denial stops this workflow; because the receiver is not reached, it does not validate the receiver's rejection path. D/A observes issuance despite the specification-side denial for the tested ordered policy.

These results apply to a controlled environment with registration and certificate-retrieval fixtures, a Flask test client, and an in-memory connection. The real consumer logic and MongoDB updates execute, but

TLS handshakes and a deployed network do not. Execution is cut off at `receive_conversation`; issuance/storage must not be described as message acceptance or downstream application execution. There is no deployment exploit claim. Nor does agreement with a modeled consequence establish Python/ProVerif refinement: the consumer run and the symbolic witness have separate identities, assumptions, and provenance. This bounded validation supplements the analysis procedure without upgrading historical observations or formal results into deployment evidence.

8.5 Generalization Beyond SAGA

SAGA is a case study, not a representative sample of all agentic systems. Programmable authorization components create consistency requirements between the policy they express, the decisions they compute, and the actions that consume those decisions. The SAGA observation motivates a verification question: when executable policy code derives a decision that another component later consumes, how is the intended policy meaning preserved across that boundary? Agent frameworks, API gateways, capability systems, and policy engines offer contexts in which this separation may warrant examination. This relevance concerns the verification question, not evidence of a shared authorization inconsistency.

Applying this perspective requires evidence connecting local policy semantics, evaluation, and enforcement under each system's assumptions. We have not verified these systems for the same inconsistency or consequence; prevalence and impact beyond SAGA remain unmeasured. Likewise, the bridge is a case-specific evidence-transfer mechanism, not a general verification framework validated for arbitrary systems.

8.6 Limitations

Implementation-to-model correspondence. For the recorded evaluation, the bridge mechanizes decision-value transfer from concrete policy input and an archived implementation observation to the executed symbolic scenario. Independent specification evaluation, provenance binding, checked-pair construction, and scenario reconstruction are completed parts of that chain. However, evaluator correctness is not theorem-proved, and its finite pattern grammar does not cover complete Python or `fnmatch` semantics. This is neither full formal verification of the matcher implementation nor correctness for arbitrary policies.

The model represents the concrete initiator by `aid_B` and the unobserved

receiver by `aid_A`, with `BuggyPolicy` as a local scenario label. These bindings remain modeling abstractions. Correspondence with deployed Provider and receiver consumers, including equivalence of their behavior to the model, is not established. General refinement between the Python implementation and the symbolic protocol remains unproved.

Deployment evidence. Direct matcher calls provide observations at the function level, not executions of a deployed service. Phase 1 adds consumer execution only in the controlled environment described above, ending at issuance/storage. No deployed-service exploit, remote attack, or malicious infrastructure experiment is demonstrated. The historical matcher archive does not identify deployed receiver or policy ownership, or establish that the matcher observation and symbolic witness describe the same concrete service session. The symbolic witness follows prescribed peer processes, without establishing arbitrary attacker control or completion of downstream tool actions. These limits qualify communication impact, not the confirmed matcher discrepancy.

Model abstraction. The private `tls_chat` channel abstracts authenticated, confidential transport rather than verifying TLS. The handoff represented by the `ProviderReleased` table is also abstract. Although issuance and acceptance are modeled, the complete token lifecycle is not: expiry, revocation, policy updates, and quota consumption are outside the evaluated properties. Concurrency is represented by replicated processes and shared token state, not by the concrete service scheduling and storage implementation. The witness can involve different issuing and accepting process instances; fresh tokens do not establish session isolation or injective agreement. The models therefore do not simulate a complete deployment or establish the correctness of these omitted behaviors. Neither D nor B includes a standalone `TokenIssue` reachability query: issuance in a witness and the acceptance-to-issuance correspondence must not be reported as such an additional query result.

Single case study. The evidence concerns SAGA and the evaluated scenarios, not deployment prevalence or current upstream vulnerability status. Bridge v1.1.1 demonstrates reproducible evidence transfer for one recorded policy evaluation, not validation over a large policy corpus. Normal allow/deny controls and the reversed-order check support the bridge evaluation; they are not a deployment-scale sample. Assessing broader applicabil-

ity requires additional agent-system studies, each with its own implementation evidence and modeling assumptions.

9 Related Work

9.1 Agent Security and Governance

Wang et al. [13] survey LLM-based autonomous agents, including their construction and evaluation. Ramchurn et al. [11] examine trust in multi-agent interactions, while AgentDojo [5] evaluates prompt injection attacks and defenses for tool-using agents. These works primarily focus on agent behavior, trust, and evaluation, informing how agent interactions can be governed. Our governance question concerns authorization decision consistency between policy evaluation and enforcement: does granting contact to an identified agent preserve the receiving user's intended policy? This differs from assessing behavioral trust or resistance to prompt injection.

9.2 Capability and Token-Based Authorization

SPKI [6] represents naming and authority in certificates. OAuth 2.0 [8] supports scoped service access. Macaroons [3] support delegation and attenuation through contextual caveats. These designs address credential representation and controlled delegation.

Credential correctness does not automatically establish authorization decision correctness. Issuance must depend on a decision consistent with the intended policy, not merely produce a credential that later passes validation. We examine that decision-to-enforcement connection in SAGA; we propose neither a new token format nor an attenuation mechanism.

9.3 Formal Verification of Authentication Protocols

ProVerif [4] and Tamarin [10] support symbolic analysis of properties including authentication correspondences and secrecy. Their results depend on the modeled behavior, assumptions, and queries; neither tool is intrinsically restricted to authentication.

Our distinction concerns verification scope: an authorization claim must include authorization-relevant decision layers or justify their connection to the model. The original queried authentication and secrecy results remain valid in our diagnostic extension. Separately, ProVerif analyzes communication consequences of the bridge-reconstructed divergence under explicit protocol assumptions. It does not execute the Python matcher or establish implementation correctness. The coverage question concerns the model-to-claim relationship, not tool soundness.

9.4 Formal Verification of Authorization Policies

Cassandra [2] specifies policy evaluation and enforcement semantics. SecPAL [1] connects declarative authorization to an evaluation strategy with soundness, completeness, and termination results under stated conditions. Margrave [7] checks policy properties and change impact for a supported subset of XACML. These works analyze policy semantics and consistency; they also address the relationship between policies, evaluation, and their surrounding software.

Our case study connects an observed implementation-level disagreement with a rule-selection contract to its modeled protocol-level consequence. This is not a competing policy language or a general evaluation-correctness result. An independent finite specification evaluator and a provenance-bound bridge now mechanize decision-value transfer for the recorded evaluation. General implementation-to-model refinement and symbolic role correspondence remain outside the evidence; the bridge is not a policy-language correctness proof or a verified compiler.

9.5 Authentication–Authorization Gaps

Authentication asks "who are you," while authorization asks "what may you do." These are distinct obligations, but their connection is longstanding: Lampson et al. [9] develop authentication theory together with distributed access control. SPKI [6] and SecPAL [1] likewise make authority and its derivation explicit. The distinction itself is not our novelty claim. These foundations leave open the question of whether a particular agent implementation preserves its intended policy semantics across protocol actions.

In SAGA, a software authorization decision can feed material release, credential issuance, and message acceptance. We study the gap between that decision's implementation and the actions consuming it. The required consistency spans policy semantics, policy evaluation implementation, and protocol enforcement. A correspondence to prior token issuance establishes credential provenance, but does not alone establish that issuance was permitted under the user's policy.

Our contribution relates a confirmed local authorization inconsistency to formal analysis of its possible communication consequence under explicit assumptions. The gate controls further distinguish the modeled release and issuance boundaries. This connection combines implementation observations, independent finite specification evaluation, checked bridge transfer, and symbolic consequence analysis. It is not a Python refinement proof or

complete matcher semantics. We neither claim to originate the study of authorization gaps nor report a deployed-service exploit. The remaining role correspondence and general refinement boundary limit the cross-layer conclusion without negating the implementation discrepancy or the completed decision transfer for the recorded case.

10 Conclusion

Policy-governed agent communication requires authorization decisions to remain consistent with intended policy semantics, their implementation evaluation, and the actions that consume them. Authentication and confidentiality provide complementary guarantees without discharging this separate obligation. Our SAGA case study analyzes a recorded authorization decision inconsistency through evidence-backed transformation and symbolic consequence analysis.

Archived function-level calls to the unmodified matcher identify an order-dependent authorization decision inconsistency. For the recorded specific-deny/general-allow rulebook, an independent finite evaluator derives Deny while the implementation observation supplies Allow. As a component of the analysis procedure, the decision bridge checks the corresponding recorded policy input and observation provenance, constructs the decision pair, and reconstructs the fixed symbolic scenario. This transformation makes the decision sources and their use in the scenario auditable while keeping implementation observation and symbolic execution distinct.

The Stage 1 diagnostic preserves the queried authentication correspondences and token secrecy without establishing the added authorization correspondence. Separately, the archived ProVerif 2.05 run of the bridge-derived model reports `ChatAccept` reachable and establishes that acceptance implies prior `TokenIssue` under the model's explicit assumptions. Independent gate controls locate distinct Provider and receiver blocking stages. These results do not transfer Stage 1 guarantees to the derived model or add a standalone issuance-reachability query to it.

The case study demonstrates why authentication and credential provenance cannot substitute for authorization-decision correctness. It combines implementation evidence, checked evidence transfer, and symbolic consequence analysis without establishing cryptographic compromise or reproducing a deployed-service exploit. The recorded policy evaluation and symbolic witness are not evidence of the same concrete service session.

The analysis is scoped to the recorded case and the stated symbolic assumptions. Evaluator correctness is not theorem-proved; the supported grammar does not cover complete Python or `fnmatch` semantics or arbitrary policies. Symbolic role binding remains an abstraction, and deployment-role correspondence and general implementation refinement remain unproved. The contribution is an evidence-backed analysis of the recorded inconsistency and its conditional symbolic consequences.

References

[1] Moritz Y. Becker, Cédric Fournet, and Andrew D. Gordon. Design and semantics of a decentralized authorization language. In *20th IEEE Computer Security Foundations Symposium (CSF)*, pages 3–15. IEEE, 2007.

[2] Moritz Y. Becker and Peter Sewell. Cassandra: Distributed access control policies with tunable expressiveness. In *5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY)*, 2004.

[3] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.

[4] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016.

[5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[6] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen. SPKI certificate theory. RFC 2693, RFC Editor, September 1999. Experimental.

[7] Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. Verification and change-impact analysis of access-control policies. In *27th International Conference on Software Engineering (ICSE)*, 2005.

[8] D. Hardt. The OAuth 2.0 authorization framework. RFC 6749, RFC Editor, October 2012. Editor; Standards Track.

[9] Butler Lampson, Martín Abadi, Michael Burrows, and Edward Wobber. Authentication in distributed systems: Theory and practice. *ACM Transactions on Computer Systems*, 10(4):265–310, 1992.

[10] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.

[11] Sarvapali D. Ramchurn, Dong Huynh, and Nicholas R. Jennings. Trust in multi-agent systems. *The Knowledge Engineering Review*, 19(1):1–25, 2004.

[12] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.

[13] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.